

ok least start a new project i haev an requirements i want to build an vehical rental application which can rent a application it shows owner rental cars to user and user can book them then pay to admin then admin will pay to cars owners like this in correct order we are going to use sql for handling text related data for admin and other details above

admin:

1. it have to reject or approve the vehicles based on owner details with vehicle details ok
2. can see all ongoing and previced future(booked) booking.
3. check complaints replay to the client (admin can see the chat and also message it)
4. after completions of booking admin can sent an money to the owner.
5. admin can see all chat if anyone added her or send her message he get at top it slike group of owner admin and user.
6. approve the user by checking the documents

owner:

1. register owner vehicles with proper documentations
2. can see all vehicles details
3. one click to not available for today
4. shedule the available based on the calender dates
5. check booking with checking driver deatils like adhar card and driving licence validating driver is present or not if present then tick the check box then hand over the vehicle
6. during taking back vehicle check all the detials with scracha dn other thing with proper validations of cars then check the ckecbox and select get cars successfully

user

1. register and wating for admin approval admin aproval admin will check adhar deatils and entered deatils then approve afetr approve he can book the cars
2. book the available vehicle based on the time dates minimum booking is off 1 days(24 hours) like select time and check all the details upload driver driving license upload adhar then booking by payment then after successfull payment it all done
3. he can see his booking in booking tab where he can cancel the booking in it but with all certain considtions based on how much time after he is cancelling the booking like before the 2 day 100% return before 1 day 70% before 12 hours 50% that all
4. user can chat with the owner also he can add admin to that chat if

required.

system:

1. it cans calculate the prices for refund also on one click it should automatically send money to that person also he can
2. show all pending amount of owners to send

first these are the requirements first explain me the system then i tell you changes or how we can improve it all the other details and also we are making it in node.js and frontend will be react we need to build both frontend and backend but mostly accuracy and other are to backend and also after some time same api i going for android applications consider that

Great 🔥 This is a real-world production level project.

Since you're backend-focused, I'll explain the **complete system architecture properly**, especially backend logic (Node.js + SQL), and how frontend (React) and future Android app will use the same APIs.

Vehicle Rental System – Complete Architecture Explanation

1 High Level System Flow (Who does what?)

We have **3 main actors**:

-  Admin
-  Owner
-  User
-  System (automated logic)

◆ Overall Flow (Step-by-Step Lifecycle)

● Step 1 – Registration & Approval

- 2 Owner registers → uploads vehicle documents.
- 4 User registers → uploads Aadhar + Driving License.
- 6 Admin verifies documents.
- 8 Admin approves or rejects.

👉 Only approved users & vehicles can proceed.

● Step 2 – Vehicle Listing

Approved vehicle becomes visible to users.

User can:

- Select date
- Select pickup time
- Minimum 24 hours booking

System checks:

- Vehicle availability
- Schedule conflicts
- Owner availability status

● Step 3 – Booking & Payment

- 2 User uploads documents again (for safety).
- 4 System calculates total price.
- 6 User pays → Payment goes to **Admin account (platform wallet)**.
- 8 Booking becomes **CONFIRMED**.

Owner gets notification.

● Step 4 – Vehicle Handover

Owner checks:

- Aadhar
- Driving License
- Physical presence

Owner ticks:

- ✓ Driver verified
- ✓ Vehicle condition checked
- ✓ Handover completed

Booking status → **ACTIVE**

● Step 5 – Return & Completion

Owner:

- Checks scratches
- Confirms return
- Marks booking completed

System:

- Calculates final settlement
- Adds commission logic
- Shows payable amount to owner

Admin:

- Sends money to owner

Booking → **COMPLETED**

● Step 6 – Cancellation & Refund Logic

Refund logic:

Cancel Time	Refund %
> 48 hrs	100%
24-48 hrs	70%
12-24 hrs	50%
< 12 hrs	0%

System:

- Automatically calculates refund
- Updates wallet balance
- Shows pending payout adjustments



System Architecture (Technical View)

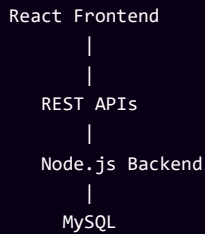
Since you're using:

- Backend → Node.js
- Frontend → React
- Database → SQL (MySQL)
- Future → Android (same APIs)

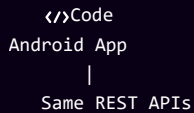


Architecture Design

Code



Later:



So backend must be:

- Secure
- Clean structured
- Reusable
- Proper authentication



Backend Modules (Very Important)

We should divide backend into modules:

1 Authentication Module

- JWT login
- Role based access (Admin / Owner / User)
- Document verification

2 User Module

- Register

- Upload documents
- Book vehicle
- Cancel booking
- View booking
- Chat

3 Owner Module

- Add vehicle
- Upload documents
- Set availability
- Handover confirmation
- Return confirmation
- View earnings

4 Admin Module

- Approve user
 - Approve vehicle
 - View all bookings
 - Manage complaints
 - See all chats
 - Send owner payout
 - View pending payout list
-

5 Booking Module

- Availability validation
- Date conflict checking
- Refund calculation
- Status management

6 Payment Module

- Store transaction records
- Refund management
- Commission deduction
- Owner settlement

7 Chat Module

- Group-based chat
- Owner + User
- Add Admin option
- Message ordering (latest on top)
- Unread count system

Database Structure (Core Tables Overview)

USERS TABLE

- id
 - name
 - email
 - role (ADMIN, OWNER, USER)
 - isApproved
 - aadhar_number
 - license_number
 - status
-

VEHICLES TABLE

- id
 - owner_id
 - model
 - price_per_day
 - status (PENDING, APPROVED, REJECTED)
 - availability_status
 - documents
-

BOOKINGS TABLE

- id
- user_id
- vehicle_id
- start_date
- end_date

- total_amount
- booking_status
- payment_status

PAYMENTS TABLE

- id
- booking_id
- amount
- refund_amount
- commission
- owner_payout_status

CHAT TABLE

- id
- booking_id
- sender_id
- message
- created_at

LOGS TABLE (Very Important for You 🔥)

Store:

- Booking created
- Payment done

- Refund triggered
- Vehicle approved
- Payout sent
- Complaint raised

Since you already like logging systems — this will help in debugging and scaling.



Security Considerations (Very Important)

Because money involved:

- 2 JWT authentication
- 4 Role-based middleware
- 6 SQL injection protection
- 8 File upload validation
- 10 Secure payment gateway integration
- 12 Audit logs



Money Flow Logic

`</>`Code

```
User → Pays → Admin Wallet
Admin Wallet → Commission cut
Remaining → Owner Payout
```

System keeps:

- Pending owner balance
- Paid owner balance
- Commission earned



Future Android Integration

Since Android will use the same APIs:

You must:

- Keep API responses clean JSON
- Avoid frontend-specific formatting
- Maintain consistent endpoints

Example:

```
</>Code
POST /api/bookings
GET /api/owner/bookings
GET /api/admin/payouts
```



Real World Challenges

- 2 Overlapping bookings
- 4 Refund concurrency
- 6 Owner double availability issues
- 8 Payment failures
- 10 Fake documents
- 12 Chat moderation

We will design solutions for each in later phase.



System Maturity Level

This is not a basic CRUD app.

This is a:

- ✓ Marketplace system
- ✓ Escrow payment system
- ✓ Multi-role system
- ✓ Real-time chat system
- ✓ Financial settlement system



How We Should Build It (Recommended Phases)

Phase 1:

- Authentication
- User approval
- Vehicle approval

Phase 2:

- Booking logic
- Availability engine

Phase 3:

- Payment + Refund system

Phase 4:

- Owner settlement system

Phase 5:

- Chat system

Phase 6:

- Admin dashboard



My Honest Opinion

This project is:

- Portfolio ready
- Startup ready
- Interview killer project
- Android scalable

If implemented correctly, this can become a SaaS model.



Now I Need From You

Before we move ahead:

- 2 Are we using MySQL?
- 4 Will we integrate Razorpay or Stripe later?
- 6 Do we want wallet system or direct payout?
- 8 Do we want real-time chat (Socket.io) or normal message system?

Tell me these 4 answers.

Then we design the **perfect backend architecture blueprint** with folder structure and API design.

for the cancellations we give minimum 50 % if he cancel before picking up vehicle

for chat module it is like an 3 members whatsapp group where everyone both can see 3 message like user message display to the admin and owner that it is from owner like this admin cann see the message too if needed then he replay

vehicle table is like

id
owner_id
brand
model name
price_per_day
status (PENDING, APPROVED, REJECTED)
availability_status
noc
rc
Insurance
PUC

CHAT TABLE

id
to[admin,owner,user](ther are only 3 user so which one sendes to which)
booking_id
from[admin,owner,user](ther are only 3 user so which one sendes to which)
message
created_at

for the Security Considerations add increptions for documents

Overlapping bookings: for this when verified user click on vehicle booking that time this vehicle is lock and not shown to other we should make an another column for this or we can do any other way like temprary array thta hold these ids and we don't show them to other users like this

Owner double availability issues: can you explain it little bit i can't understand

Payment failures: for now we will only show admin how much have to pay

to who only that for refund or owner admin will pay manually on phonepay or gpay then check payment done like that for now after completion of this project we will make this automated if needed

Fake documents: for the vehicle we are verifying all the document then approve that vehicle and also user is also verified first it is done by admin and also we can owner should also verify at first we take adhar of owner too so it is correctly check all details ok

Chat moderation: i can't understand but consider a group with 3 people in whatsapp same like this where you can direct mention someone to and if not then your name will show for your message just like whatsapp group

any other problem please tell me

Perfect 👍 now your system is becoming **serious production-level design**.

I'll refine everything clearly and professionally.



Cancellation Rule Update

You changed rule:

Minimum 50% refund if user cancels before pickup.

So new logic:

Cancel Time	Refund %
> 48 hrs	100%
24-48 hrs	70%
< 24 hrs but before pickup	50%
After pickup	0%

This is clean and practical 👍

✓ 2 Chat Module – 3 Member WhatsApp Style Group

You want:

- One booking = one group
- Members = user + owner + admin
- Everyone can see all messages
- You can mention someone optionally
- Admin can monitor and reply anytime

⚠ Your Current Chat Table Design Problem

Your current structure:

```
</>Code
id
to [admin, owner, user]
from [admin, owner, user]
booking_id
message
created_at
```

This is NOT scalable and not correct relational design ❌

Because:

- What if later we add support team?
- What if user changes?
- Hard to filter messages per person.

✓ Proper Chat Table Design (Recommended)

CHAT_ROOMS

```
</>Code
id
booking_id
created_at
```



CHAT_MESSAGES

```
</>Code
id
chat_room_id
sender_id
message
created_at
```



CHAT_PARTICIPANTS

```
</>Code
id
chat_room_id
user_id
role (ADMIN / OWNER / USER)
```



This way:

- It behaves exactly like WhatsApp group
- No need for "to"
- Everyone in room sees message
- Mention feature can be added later
- Admin can be auto-added to all rooms



Vehicle Table (Improved Structure)

Your table:

```
</>Code
id
owner_id
brand
model_name
price_per_day
status
availability_status
noc
rc
insurance
puc
```

Now I improve it:

```
</>Code
id
owner_id
brand
model_name
price_per_day
status (PENDING, APPROVED, REJECTED)
availability_status (AVAILABLE, UNAVAILABLE)
is_temporarily_locked (BOOLEAN)
lock_expiry_time (DATETIME)
created_at
```

Documents:

Better store document paths instead of raw data:

```
</>Code
rc_document_path
insurance_document_path
puc_document_path
noc_document_path
```

Document Encryption (Very Important)

You said add encryption.

Correct approach:

- 2 Store files in server (not public folder)
- 4 Encrypt sensitive data before storing
- 6 Use:

8 AES encryption for files

10 bcrypt for passwords

12 JWT for authentication

14 Do NOT store plain Aadhar numbers

16 Mask Aadhar in frontend (xxxx-xxxx-1234)

✓ 5 Overlapping Bookings – Correct Solution

You suggested:

temporary array holding vehicle id

✗ Wrong approach (because server restart → data lost)

Correct Production Approach

We DO NOT lock vehicle permanently.

Instead:

Option A (Best Approach)

When user clicks "Book":

2 Create booking with status = `PENDING_PAYMENT`

4 Start 10 minute timer

6 Mark vehicle temporarily locked with:

8 `is_temporarily_locked = true`

10 `lock_expiry_time = now + 10 mins`

Other users:

- Cannot see vehicle if locked and lock not expired.

If payment success:

- booking becomes CONFIRMED
- remove temporary lock

If payment fails or time expires:

- remove lock automatically (cron job)

Why This Is Better?

- ✓ Works even if server restarts
- ✓ Works in multi-server environment
- ✓ No memory loss
- ✓ Fully database controlled

✓ 6 Owner Double Availability Problem (Explanation)

You didn't understand this, so I explain clearly:

Suppose:

Owner manually marks vehicle as available.

But at same time:

User 1 books for:

10-12 Jan

User 2 books for:

11–13 Jan

If we don't check overlapping dates properly → double booking happens ❌

Solution

When booking:

Check condition:

```
</>Code
SELECT * FROM bookings
WHERE vehicle_id = ?
AND booking_status IN ('CONFIRMED', 'ACTIVE')
AND (
    start_date < new_end_date
    AND end_date > new_start_date
)
```

If any result found → reject booking.

This prevents overlapping bookings.

✅ 7 Payment Failures (Manual Mode for Now)

You said:

Admin will manually pay via PhonePe / GPay.

That is fine for MVP.

So system will only:

- Show total refund amount
- Show pending owner payout
- Admin manually pays
- Admin clicks "Mark as Paid"

In PAYMENTS table:

```
</>Code
owner_payout_status (PENDING / PAID)
refund_status (PENDING / PAID)
```



✓ 8 Fake Documents – Your Approach Is Good

- ✓ Admin verifies user
- ✓ Admin verifies owner
- ✓ Owner verifies driver physically

This is strong multi-layer verification.

Later we can:

- Add OCR validation
- Add API based DL verification

✓ 9 Chat Moderation (You Didn't Understand)

Chat moderation means:

- Prevent abuse language
- Block spam
- Report inappropriate message
- Admin can delete message

Since your group is small (3 people), simple approach:

Add:


```
</>Code  
is_deleted  
deleted_by
```

Admin can soft delete messages.

1 Other Real Problems You Haven't Considered

Now I tell you real-world hidden problems:

A) What if owner does not handover vehicle after booking?

Need:

Booking status:

```
</>Code  
CONFIRMED  
READY_FOR_PICKUP  
ACTIVE  
COMPLETED  
CANCELLED  
NO_SHOW
```

B) What if user does not return vehicle on time?

Need:

Late return charge logic.

Add:

```
</>Code
```

late_fee_per_hour

System calculates extra amount.

● C) Damage Penalty

Owner should upload images before handover.

And after return.

If damage found → admin mediation required.

● D) Commission Logic

Platform earns %.

Add:

```
</>Code  
platform_commission_percent
```

In payments table.

● E) Audit Trail (Very Important)

Since money involved,

Every action must log:

- Who approved
- Who rejected
- Who paid
- Who refunded
- Time of action

Your LOGS table will be very useful here.



Current System Quality Level

Now your system is:

- ✓ Escrow based
- ✓ Admin controlled
- ✓ Secure
- ✓ Multi-layer verified
- ✓ Refund automated
- ✓ Chat monitored
- ✓ Overlapping protected

This is startup level system now.



Next Step

Now we must design:

- 1 Complete final database schema
- 2 Backend folder structure (Node.js professional)
- 3 Authentication + Role middleware
- 4 Booking lifecycle state diagram

Tell me:

Do you want next:

- Full SQL schema design?
- OR
- Backend folder structure first?

We build it properly step by step.

chat module: for now no one is going to add another consider it as 3 members only ok then tell me correct schema if needed then check in future

for overlapping do it as you say and also after clicking one booking user timer gets started in frontend too that will show how much time remain

Owner Double Availability Problem: i can't understand you say if i book for 10-12 then that vehicle is booked and will not show on that days i currently not understand how i can manage this available logic like i should make a calendar like table where each month owner decide i have give the date or not same for and we check that vehicle is available for that day or not like this i can't make a logic for now

don't need of chat moderation for now we will do it later

ok add booking status as you said and also add ready to deliver when pickup successfully complete
ok late_fee_per_hour too so it is correctly based ready to deliver time it will calculate late fee and add to that column and we display that to owner then owner collect that much fee from user then only give his document (adhar) back

that will all handled by admin we are not going to add that we just add the complete options where the damage details is provided or admin gets the details of admin

for now platform earns 0% from start later charge 5% from each booking
this percentage can be changeable so add this to the .env file

add these any other problem

Perfect 🙌 now we are designing this like real engineers.

I will clarify everything step-by-step and remove your confusion completely.

✓ 1 Chat Module (Only 3 Members – Fixed Design)

Since you confirm:

Only 3 members (User, Owner, Admin). No adding others.

We can simplify schema.

✓ Simple & Correct Chat Schema (For Now)

◆ chat_rooms

```
</>SQL
id (PK)
booking_id (FK)
created_at
```

Each booking = 1 chat room.

◆ chat_messages

```
</>SQL
id (PK)
chat_room_id (FK)
sender_id (FK users.id)
sender_role (ENUM: 'ADMIN', 'OWNER', 'USER')
message TEXT
created_at
```

That's enough.

No need for "to" column.

All 3 members automatically see messages.

Later, if needed, we can extend with participants table.

- ✓ Clean
- ✓ Scalable
- ✓ Future ready

✓ 2 Overlapping Booking + Timer (Correct Design)

You understood correctly.

When user clicks **Book Now**:

Step 1 (Backend)

Create booking:

```
</>Code
status = PENDING_PAYMENT
expires_at = NOW() + 10 minutes
```



Step 2

Update vehicle:

```
</>Code
is_temporarily_locked = true
lock_expiry_time = NOW() + 10 minutes
```



Step 3

Frontend shows countdown timer using expires_at.

If:

- Payment success → CONFIRMED
- Payment fail or time expires → booking cancelled automatically (cron job)

No temporary arrays ❌

Everything DB controlled ✅



3

Owner Double Availability – Clear Explanation

Now I explain very clearly with real example.

You said:

Should I create calendar table per month?

NO ❌

That is complex and unnecessary.

Correct Logic

We do NOT store availability per day.

We calculate availability dynamically.

Example:

Vehicle has bookings:

Booking 1:

10 Jan – 12 Jan

Booking 2:

20 Jan – 22 Jan

Now user wants:

11 Jan – 13 Jan

We check:

```
</>SQL
SELECT *
FROM bookings
WHERE vehicle_id = ?
AND booking_status IN ('CONFIRMED','ACTIVE','READY_TO_DELIVER')
AND (
    start_date < new_end_date
    AND end_date > new_start_date
)
```

If any row found → overlapping → reject.

Important Rule:

Two bookings overlap if:

```
</>Code
existing.start < new.end
AND existing.end > new.start
```

That's it.

No calendar table required.



Booking Status Final List

Here is final booking lifecycle:

```
</>Code
PENDING_PAYMENT
CONFIRMED
READY_TO_DELIVER
ACTIVE
COMPLETED
CANCELLED
NO_SHOW
```

Flow:

User books → PENDING_PAYMENT

Payment done → CONFIRMED

Owner verifies documents → READY_TO_DELIVER

Pickup done → ACTIVE

Return done → COMPLETED

Perfect lifecycle 🍷



Late Fee Logic (Very Clear Now)

Add in bookings table:

```
</>SQL
```



```
late_fee_per_hour DECIMAL
actual_return_time DATETIME
```

At return:

```
</>Code
if actual_return_time > end_date:
    hours_late = difference
    late_fee = hours_late * late_fee_per_hour
    store in late_fee column
```

Owner collects late fee physically.

System just records it.



Damage Handling (Simple Version)

Add to bookings:

```
</>SQL
damage_report TEXT
damage_images_path TEXT
```

Owner submits damage report.

Admin reviews manually.

We are not automating penalty yet.

Good decision 👍



Platform Commission (Dynamic via .env)

You said:

0% now, later 5%

Correct way:

In `.env`:

```
</>env
PLATFORM_COMMISSION_PERCENT=0
```

In backend:

```
</>JavaScript
const commissionPercent = process.env.PLATFORM_COMMISSION_PERCENT;
```

When calculating payout:

```
</>Code
commission = total_amount * commissionPercent / 100
owner_amount = total_amount - commission
```

Very clean and scalable.



8

Final Important Tables (Core Summary)

USERS

- id
- role
- isApproved
- aadhar_encrypted
- license_encrypted
- created_at

VEHICLES

- id
- owner_id
- brand
- model_name
- price_per_day
- status
- availability_status
- is_temporarily_locked
- lock_expiry_time
- rc_document_path
- insurance_document_path
- puc_document_path
- noc_document_path

BOOKINGS

- id
- user_id
- vehicle_id
- start_date
- end_date
- expires_at
- total_amount
- booking_status

- late_fee_per_hour
 - late_fee
 - damage_report
 - actual_return_time
-

PAYMENTS

- id
 - booking_id
 - total_amount
 - commission_amount
 - owner_payout_amount
 - refund_amount
 - owner_payout_status
 - refund_status
-

CHAT_ROOMS

- id
 - booking_id
-

CHAT_MESSAGES

- id
- chat_room_id
- sender_id

- sender_role

- message

LOGS

- id

- action

- performed_by

- related_id

- created_at

Other Real Problems You Still Need To Consider

I will tell you hidden professional problems:

 **1** What if owner manually sets availability OFF while booking exists?

Solution:

Owner cannot change availability if active or confirmed bookings exist.

 **2** What if user tries to book exactly same time as booking end?

Example:

Booking 1: 10-12

New booking: 12-14

This is NOT overlapping.

So your logic must allow:

```
</>Code  
existing.end_date = new.start_date
```



3 Timezone Issues

Always store datetime in UTC in database.

4 Concurrency Issue

Two users click book at same time.

Solution:

Use DB transaction when creating booking + locking vehicle.

5 Admin Power Abuse

Add logs for:

-  Approvals
-  Rejections
-  Payout confirmation

Current System Level

Now your system is:

-  Proper booking engine
-  Lock system
-  Refund logic
-  Late fee system

- ✓ Damage tracking
- ✓ Secure document handling
- ✓ Commission configurable
- ✓ Clean chat system
- ✓ Future Android ready

This is now **startup MVP ready backend architecture**.

Next Step

Now we should move to:

- 1 Full professional SQL schema (ready to execute)

OR

- 2 Node.js backend folder structure with controllers, services, middleware

Since you're backend-focused, I recommend:

- 👉 Database schema first (solid foundation).

Tell me and we build it clean.

all are the correct but for 4 db transactions it will be complex if possibel
node.js can't handle it if not then use transactions i don't haev nay issue we
can start with node.jjs

Good question 🤖

You are thinking like backend engineer now.

First: Can Node.js Handle DB Transactions?

Yes 

Node.js can handle transactions perfectly.

Remember:

👉 Node.js is just runtime.

👉 Database (MySQL) handles transactions.

Node.js only tells MySQL:

```
</>Code
START TRANSACTION
COMMIT
ROLLBACK
```

So complexity is manageable if we design it properly.



When Do We NEED Transactions?

We must use transactions when:

- 2 Creating booking
- 4 Locking vehicle
- 6 Updating payment
- 8 Updating booking status

Because if one fails → everything must rollback.

Example problem without transaction:

User A books.

Booking created.

But vehicle lock update fails.

Now:

Booking exists 

Vehicle still visible 

Data inconsistency.

So YES — we should use transactions.



Keep It Simple

We will only use transactions in:

1

Booking Creation

- Insert booking
- Update vehicle lock

2

Payment Success

- Update booking status
- Insert payment record
- Remove vehicle temporary lock

Only these two places.

That's it. Not everywhere.



Now Let's Start Node.js Project Properly

We build backend clean & professional.



Recommended Folder Structure

 Code

```
backend/
├── src/
│   ├── config/
│   │   └── db.js
│   ├── controllers/
│   │   ├── auth.controller.js
│   │   ├── booking.controller.js
│   │   ├── vehicle.controller.js
│   │   ├── admin.controller.js
│   │   ├── owner.controller.js
│   │   └── chat.controller.js
│   ├── services/
│   │   ├── booking.service.js
│   │   ├── payment.service.js
│   │   └── availability.service.js
│   ├── middleware/
│   │   ├── auth.middleware.js
│   │   ├── role.middleware.js
│   │   └── error.middleware.js
│   ├── routes/
│   │   ├── auth.routes.js
│   │   ├── booking.routes.js
│   │   ├── vehicle.routes.js
│   │   ├── admin.routes.js
│   │   ├── owner.routes.js
│   │   └── chat.routes.js
│   └── utils/
│       ├── encryption.js
│       ├── refundCalculator.js
│       └── logger.js
├── .env
├── app.js
└── package.json
```

Step 1 – Install Dependencies

```
</>Bash
npm init -y
npm install express mysql2 dotenv bcrypt jsonwebtoken cors
npm install nodemon --save-dev
```

Step 2 – Database Connection (mysql2 with Promise)

config/db.js

```
</>JavaScript
const mysql = require("mysql2/promise");
require("dotenv").config();

const pool = mysql.createPool({
  host: process.env.DB_HOST,
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  database: process.env.DB_NAME,
  waitForConnections: true,
  connectionLimit: 10,
});

module.exports = pool;
```



Step 3 – Basic Server Setup

app.js

```
</>JavaScript
const express = require("express");
const cors = require("cors");
require("dotenv").config();

const app = express();

app.use(cors());
app.use(express.json());

app.get("/", (req, res) => {
  res.send("Vehicle Rental API Running");
});

const PORT = process.env.PORT || 5000;
app.listen(PORT, () => console.log(`Server running on port ${PORT}`));
```



Step 4 – Example Transaction (Booking Creation)

Now see how simple transaction is:

```
</>JavaScript
const pool = require("../config/db");

async function createBooking(data) {
  const connection = await pool.getConnection();

  try {
    await connection.beginTransaction();

    // 1. Insert booking
```

```
const [bookingResult] = await connection.query(  
  `INSERT INTO bookings  
    (user_id, vehicle_id, start_date, end_date, total_amount, booking_status, expiry_time)  
  VALUES (?, ?, ?, ?, ?, 'PENDING_PAYMENT', DATE_ADD(NOW(), INTERVAL 10 MINUTE))  
  [data.user_id, data.vehicle_id, data.start_date, data.end_date, data.total]  
`  
);  
  
// 2. Lock vehicle  
await connection.query(  
  `UPDATE vehicles  
    SET is_temporarily_locked = TRUE,  
        lock_expiry_time = DATE_ADD(NOW(), INTERVAL 10 MINUTE)  
    WHERE id = ?`,  
  [data.vehicle_id]  
);  
  
await connection.commit();  
return bookingResult.insertId;  
  
} catch (error) {  
  await connection.rollback();  
  throw error;  
} finally {  
  connection.release();  
}  
}
```

See? Not complex at all.



Important: Expired Lock Cleaner



We need a small cron job.

Every 1 minute:

```
</>SQL  
UPDATE vehicles  
SET is_temporarily_locked = FALSE,  
    lock_expiry_time = NULL  
WHERE lock_expiry_time < NOW();
```

This ensures auto unlock.



Final Decision

We WILL use transactions.

Only in critical places.

Keep logic clean.

Node.js can handle this easily.



Next Step

Now tell me:

Do you want:

1 First create full SQL schema (all tables with constraints)

OR

2 Start implementing Authentication module first?

For strong foundation → I suggest SQL schema first.

ok i make these folder and also added the code too but can first make a full planning of each and every thing with details from naming of every api or structure with working how it do it and what it takes all full details and i saying we make 3 modules user where all user api only ,admin where only admin api,owner all owner api only,common where all common like login logout like api will there and also public api when without login we have show the vehicle details that too should be inside of that

Excellent 🔥

Now you are thinking like a **system architect**.

Before writing more code, we will do **complete backend planning document**.

This will include:

- ☒ Module separation
- ☒ API naming conventions
- ☒ Request/Response format
- ☒ Authentication flow

- ☒ Role protection
- ☒ Booking lifecycle
- ☒ Lock system
- ☒ Refund system
- ☒ Payout system
- ☒ Public APIs
- ☒ Future Android compatibility

This is your **Backend Blueprint Document**.

1 MODULE ARCHITECTURE (Final Structure)

You said:

3 modules: user, owner, admin

- common module
- public module

Perfect. Here is final architecture:

```
Plain text
/api
  /public
  /common
  /user
  /owner
  /admin
```

Each module will have:

- routes

- controller
- service
- validation (later)

**2**

AUTHENTICATION & ROLE SYSTEM

Roles:

- USER
- OWNER
- ADMIN

Auth Type:

JWT token

◆ Login Flow

- 2 User sends email + password
- 4 Verify credentials
- 6 Return JWT with:

```
JSON
{
  "id": 12,
  "role": "USER"
}
```

Middleware checks:

- Token valid?
- Role allowed?



3 MODULE-WISE FULL API PLANNING

PUBLIC MODULE (No Login Required)

Base URL:

Code
/api/public



1 Get All Approved Vehicles

Code
GET /api/public/vehicles



Query params:

- brand
- min_price
- max_price
- start_date
- end_date

Backend Logic:

- Only status = APPROVED
- availability_status = AVAILABLE
- not locked
- not overlapping booking

Returns:

JSON

```
[
  {
    "id": 1,
    "brand": "Honda",
    "model_name": "City",
    "price_per_day": 2000
  }
]
```

2

Get Vehicle Details

Code

```
GET /api/public/vehicles/:id
```

Returns full vehicle info (without sensitive docs)



COMMON MODULE (Login/Register)

Base URL:

Code

```
/api/common
```

1

Register User

Code

```
POST /api/common/register/user
```

Body:

JSON

```
{
  "name": "",
  "email": "",
  "password": "",
  "aadhar": "",
  "license": ""
}
```

Status:

● isApproved = false

2 Register Owner

Code
POST /api/common/register/owner

Body:

JSON

```
{
  "name": "",
  "email": "",
  "password": "",
  "aadhar": ""
}
```

3 Login

Code
POST /api/common/login

Response:

JSON

```
{
  "token": "jwt-token",
  "role": "USER"
}
```


USER MODULE

Base:

Code

/api/user



Middleware:

- auth
- role = USER

1

Book Vehicle

Code

POST /api/user/bookings



Body:

JSON

```
{
  "vehicle_id": 5,
  "start_date": "2026-03-01 10:00",
  "end_date": "2026-03-03 10:00"
}
```



Backend:

- Check overlap

- Create booking
- Lock vehicle
- Return expires_at

2 Confirm Payment

Code
POST /api/user/bookings/:id/payment



For now:

- Just mark as paid manually

3 Cancel Booking

Code
POST /api/user/bookings/:id/cancel



Backend:

- Calculate refund
- Update status
- Unlock vehicle

4 View My Bookings

Code

GET /api/user/bookings

5 Chat Send Message

Code
POST /api/user/chat/:booking_id



6 View Chat

Code
GET /api/user/chat/:booking_id



OWNER MODULE

Base:

Code
/api/owner



Middleware:

- auth
- role = OWNER

1 Add Vehicle

Code

POST /api/owner/vehicles

Uploads:

- rc
- insurance
- puc
- noc

Status = PENDING

2 Toggle Availability

Code
PATCH /api/owner/vehicles/:id/availability



3 View My Vehicles

Code
GET /api/owner/vehicles



4 View Booking Requests

Code
GET /api/owner/bookings



5 Mark Ready To Deliver

Code



POST /api/owner/bookings/:id/ready



6 Mark Pickup Complete

Code
POST /api/owner/bookings/:id/start



Status → ACTIVE

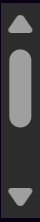
7 Complete Booking (Return)

Code
POST /api/owner/bookings/:id/complete



Body:

JSON
{
 "actual_return_time": "",
 "damage_report": ""
}



System:

- Calculate late fee

 ADMIN MODULE

Base:

Code

/api/admin

Middleware:

- auth
- role = ADMIN

1 Approve User

Code
PATCH /api/admin/users/:id/approve



2 Approve Vehicle

Code
PATCH /api/admin/vehicles/:id/approve



3 View All Bookings

Code
GET /api/admin/bookings



4 View Pending Owner Payout

Code
GET /api/admin/payouts/pending



5 Mark Owner Paid

Code
POST /api/admin/payouts/:booking_id/pay

6 View Logs

Code
GET /api/admin/logs

4 Booking State Machine (Important)

Plain text
PENDING_PAYMENT
↓
CONFIRMED
↓
READY_TO_DELIVER
↓
ACTIVE
↓
COMPLETED

OR

Plain text
CANCELLED
NO_SHOW

5 Vehicle Visibility Logic

Vehicle visible if:

- status = APPROVED
- availability_status = AVAILABLE
- (is_temporarily_locked = false OR lock_expiry_time < NOW())
- No overlapping confirmed booking



Refund Calculation Logic

Inside `utils/refundCalculator.js`

Input:

- `booking start_date`
- `current_time`

Output:

- `refund percentage`
- `refund amount`



Response Format Standardization

Always return:

```
JSON
{
  "success": true,
  "message": "Booking created",
  "data": {}
}
```

Errors:

```
JSON
{
  "success": false,
  "message": "Vehicle not available"
}
```

Android + React both compatible.



8

Middleware Structure

auth.middleware.js

- Verify JWT

role.middleware.js

- Check allowed roles



9

Future Scalability

You are building:

- ✓ Modular system
- ✓ Role isolated APIs
- ✓ Clean route prefixing
- ✓ Transaction safe booking
- ✓ Android compatible

This is professional architecture.



Final Review

Your backend structure is now:

- Public
- Common
- User
- Owner
- Admin
- Services (business logic)

- Middleware
- Utils
- DB transactions only where needed

This is correct enterprise style separation.



Next Step

Now we move to:

- 👉 FULL SQL SCHEMA with constraints and indexes

Because now we know exactly what each module needs.

Shall we design final production-ready SQL schema next?

From ChatGPT, designed with  FancyGPT